TEMA 6. Genericidad

1. Empleando `void*` en C o `Object` en Java, pon un ejemplo de una estructura de datos, que empleando un array primitivo, permita alojar cualquier tipo de dato.

Respuesta

En C, se puede utilizar el tipo `void*` para crear una estructura de datos genérica que permita almacenar cualquier tipo de dato. Por ejemplo, se puede implementar un array que almacene punteros a `void`, y luego realizar conversiones explícitas al tipo adecuado al acceder a los elementos. En Java, se puede lograr algo similar utilizando el tipo `Object`, ya que todos los tipos en Java son subtipos de `Object`. Sin embargo, en ambos casos, se pierde el chequeo de tipos en tiempo de compilación, lo que puede llevar a errores en tiempo de ejecución si no se realizan las conversiones correctamente.

Ejemplo en C:

```c
#include <stdio.h>
#include <stdlib.h>

void* array[10];

int main() {
    int a = 5;
    double b = 3.14;
    char c = 'x';

    array[0] = &a;
    array[1] = &b;
    array[2] = &c;

    printf("%d\n", *(int*)array[0]);
    printf("%f\n", *(double*)array[1]);
    printf("%c\n", *(char*)array[2]);

    return 0;
}
```

Ejemplo en Java:

```java
Object[] array = new Object[10];

array[0] = 5; // Autoboxing de int a Integer
array[1] = 3.14; // Autoboxing de double a Double
array[2] = 'x'; // Autoboxing de char a Character

System.out.println((Integer) array[0]);
System.out.println((Double) array[1]);
System.out.println((Character) array[2]);
```

Ambos ejemplos muestran cómo se pueden almacenar diferentes tipos de datos en una estructura genérica, pero requieren conversiones explícitas al acceder a los elementos.

2. Brevemente, ¿Qué significa la **programación genérica**? ¿Es el ejemplo anterior un ejemplo básico de programación genérica?

Respuesta

La programación genérica es un paradigma de programación que permite escribir código que puede trabajar con cualquier tipo de dato, sin necesidad de especificar el tipo exacto en el momento de escribir el código. Esto se logra mediante el uso de parámetros de tipo, que actúan como marcadores de posición para los tipos concretos que se utilizarán en tiempo de compilación. Este enfoque mejora la reutilización del código y la seguridad de tipos, ya que los errores relacionados con tipos se detectan en tiempo de compilación.

El ejemplo anterior, aunque permite almacenar diferentes tipos de datos en una estructura, no es un ejemplo de programación genérica en el sentido estricto. En C, el uso de `void*` y en Java el uso de `Object` no proporcionan seguridad de tipos en tiempo de compilación, ya que las conversiones de tipo deben realizarse manualmente. Por lo tanto, no aprovechan las ventajas completas de la programación genérica, como el chequeo de tipos en tiempo de compilación y la eliminación de la necesidad de conversiones explícitas.

3. Indica los problemas respecto al chequeo de tipos, de emplear `void*` o `Object` cuando se crean estructuras de datos genéricas.

Respuesta

El principal problema de utilizar `void*` en C o `Object` en Java para crear estructuras de datos genéricas es la falta de seguridad de tipos en tiempo de compilación. Esto significa que el compilador no puede verificar si los tipos de datos utilizados son correctos, lo que puede dar lugar a errores en tiempo de ejecución si se realizan conversiones incorrectas.

En el caso de `void*` en C, el programador debe realizar conversiones explícitas al tipo adecuado al acceder a los elementos de la estructura. Si la conversión es incorrecta, el comportamiento del programa es indefinido, lo que puede provocar fallos difíciles de depurar. Además, no hay forma de garantizar que los datos almacenados en la estructura sean del tipo esperado.

En Java, el uso de `Object` presenta problemas similares. Aunque Java lanza una excepción `ClassCastException` si se intenta realizar una conversión inválida, esto ocurre en tiempo de ejecución, lo que puede hacer que los errores sean más difíciles de detectar y corregir. Además, el uso de `Object` requiere realizar conversiones explícitas, lo que aumenta la complejidad del código y reduce su legibilidad.

4. Vamos entonces con mecanismos de mejora de la programación genérica ¿Qué son los **parámetros de tipo**?

Respuesta

Los parámetros de tipo son una característica de la programación genérica que permite definir clases, interfaces y métodos que pueden trabajar con diferentes tipos de datos sin necesidad de especificar los tipos concretos en el momento de escribir el código. En lugar de utilizar un tipo específico, se utiliza un parámetro de tipo como marcador de posición, que se sustituye por un tipo concreto en tiempo de compilación.

Por ejemplo, en Java, se pueden definir parámetros de tipo utilizando la notación `<T>`, donde `T` es un nombre arbitrario que representa el tipo genérico. Esto permite escribir código más flexible y reutilizable, ya que la misma clase o método puede trabajar con diferentes tipos de datos sin necesidad de duplicar el código.

Un ejemplo sencillo sería una clase genérica `Caja` que puede almacenar cualquier tipo de objeto:

```
public class Caja<T> {
    private T contenido;

    public Caja(T contenido) {
        this.contenido = contenido;
    }

    public T getContenido() {
        return contenido;
    }
}
```

En este caso, `T` es un parámetro de tipo que se sustituye por un tipo concreto cuando se instancia la clase, como `Caja<String>` o `Caja<Integer>`.

5. En Java existe "generics", en C++ existen "templates". Pon un ejemplo de uso de programación genérica en ambos, instanciando una lista o vector dinámico que solo admite `String`. Introduce valores, y luego haz un recorrido de ellos mostrando cómo cada elemento es del tipo concreto con seguridad.

Respuesta

En Java, los generics permiten definir estructuras de datos que trabajan con tipos específicos, proporcionando seguridad de tipos en tiempo de compilación. En C++, los templates cumplen una función similar, permitiendo definir clases y funciones genéricas que se instancian con tipos concretos en tiempo de compilación.

Ejemplo en Java:

```java
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<String> lista = new ArrayList<>();
        lista.add("Hola");
        lista.add("Mundo");

        for (String elemento : lista) {
            System.out.println(elemento);
        }
    }
}
```

Ejemplo en C++:

```cpp
#include <iostream>
#include <vector>
#include <string>

int main() {
    std::vector<std::string> lista;
    lista.push_back("Hola");
    lista.push_back("Mundo");

    for (const std::string& elemento : lista) {
        std::cout << elemento << std::endl;
    }

    return 0;
}
```

En ambos casos, las estructuras de datos están restringidas al tipo `String` (o `std::string` en C++), lo que garantiza que todos los elementos sean del tipo esperado y elimina la necesidad de realizar conversiones explícitas.

6. Sobre el funcionamiento de la programación genérica. ¿Qué hace el compilador cuando se instancia una clase que tiene parámetros de tipo? ¿Hace lo mismo C++ y Java? ¿Qué es el "type erasure" de Java y la "instanciación de plantillas" de C++?

Respuesta

Cuando se instancia una clase con parámetros de tipo en Java, el compilador realiza un proceso llamado "type erasure". Este proceso elimina los parámetros de tipo en tiempo de compilación y los reemplaza con su tipo base (generalmente `Object`). Esto significa que, aunque el código genérico proporciona seguridad de tipos en tiempo de compilación, en tiempo de ejecución no hay información sobre los tipos genéricos. Por ejemplo, `List<String>` y `List<Integer>` se tratan como el mismo tipo en tiempo de ejecución.

En C++, el mecanismo es diferente. Cuando se utiliza una clase o función genérica con templates, el compilador genera una versión específica del código para cada tipo utilizado. Este proceso se llama

"instanciación de plantillas" y ocurre en tiempo de compilación. Como resultado, en C++ se genera código específico para cada tipo, lo que permite optimizaciones y mantiene la información de tipos en tiempo de ejecución.

En resumen, Java utiliza "type erasure" para mantener compatibilidad con versiones anteriores y reducir el tamaño del código, mientras que C++ genera código específico para cada tipo, lo que puede aumentar el tamaño del binario pero proporciona mayor flexibilidad y eficiencia.

7. Vamos a crear una nueva clase con parámetros de tipo. Define en Java una clase `Par`, que permite alojar dos valores de tipos diferentes. Incluye un constructor y un getter para cada tipo. Pon un ejemplo de uso de ese `Par`, por ejemplo para especificar el tipo de retorno de una función que devuelve en un `Par` la media y desviación típica de un array de `double`.

Respuesta

La clase `Par` puede definirse utilizando parámetros de tipo para alojar dos valores de tipos diferentes. A continuación, se muestra su implementación y un ejemplo de uso:

```java
public class Par<T, U> {
    private final T primero;
    private final U segundo;

    public Par(T primero, U segundo) {
        this.primero = primero;
        this.segundo = segundo;
    }

    public T getPrimero() {
        return primero;
    }

    public U getSegundo() {
        return segundo;
    }
}

public class Main {
    public static Par<Double, Double> calcularMediaYDesviacion(double[] numeros) {
        double suma = 0;
        for (double num : numeros) {
            suma += num;
        }
        double media = suma / numeros.length;

        double sumaDesviaciones = 0;
        for (double num : numeros) {
            sumaDesviaciones += Math.pow(num - media, 2);
        }
        double desviacion = Math.sqrt(sumaDesviaciones / numeros.length);

        return new Par<>(media, desviacion);
    }

    public static void main(String[] args) {
        double[] numeros = {1.0, 2.0, 3.0, 4.0, 5.0};
        Par<Double, Double> resultado = calcularMediaYDesviacion(numeros);

        System.out.println("Media: " + resultado.getPrimero());
        System.out.println("Desviación típica: " + resultado.getSegundo());
```

```
    }
}
```

En este ejemplo, la clase `Par` se utiliza para devolver dos valores relacionados (la media y la desviación típica) de forma segura y genérica.

8. En Java, se pueden declarar parámetros de tipo también a nivel de método, no solo a nivel de clase. Pon un ejemplo con un método genérico `seleccionaUno`, que pasados dos objetos del mismo tipo, te devuelva aleatoriamente uno de ellos. Muestra la diferencia de definirlo con dos `Object`, a definirlo con dos parámetros de tipo, en términos de (i) evitar downcasting y (ii) forzar que ambos objetos sean del mismo tipo.

Respuesta

Un método genérico puede definirse utilizando parámetros de tipo a nivel de método. Esto permite trabajar con tipos específicos sin necesidad de definir una clase genérica completa. A continuación, se muestra un ejemplo del método `seleccionaUno`:

```java
import java.util.Random;

public class Util {
    public static <T> T seleccionaUno(T a, T b) {
        Random random = new Random();
        return random.nextBoolean() ? a : b;
    }

    public static Object seleccionaUnoConObject(Object a, Object b) {
        Random random = new Random();
        return random.nextBoolean() ? a : b;
    }

    public static void main(String[] args) {
        // Uso con genéricos
        String resultado1 = seleccionaUno("Hola", "Mundo");
        System.out.println("Resultado genérico: " + resultado1);

        // Uso con Object (requiere downcasting)
        Object resultado2 = seleccionaUnoConObject("Hola", 42);
        System.out.println("Resultado con Object: " + resultado2);
    }
}
```

Diferencias:

1. **Evitar downcasting**: Con genéricos, el tipo de retorno es el mismo que el de los parámetros, por lo que no es necesario realizar conversiones explícitas. En el caso de `Object`, se debe realizar un downcasting para trabajar con el tipo específico, lo que puede provocar errores en tiempo de ejecución.
2. **Forzar que ambos objetos sean del mismo tipo**: Con genéricos, el compilador garantiza que ambos parámetros sean del mismo tipo. En el caso de `Object`, no hay tal restricción, lo que permite pasar objetos de tipos diferentes, lo que puede ser problemático.

9. ¿Se pueden establecer restricciones en los parámetros de tipo? Por ejemplo, si quiero definir un tipo genérico `<T>`, ¿puedo decir que tenga que ser, al menos, un número para poder tratarlo como tal? Pon un ejemplo en Java de un `Punto` con dos

coordenadas, métodos `getX`, `getY`, y una función `calcularDistanciaA` otro `Punto`. Permite que esas coordenadas sean cualquier tipo de número. Pon dos soluciones: una simplemente creando coordenadas de tipo `Number` y otra añadiendo generics para reforzar el chequeo de tipos y saber exactamente con qué tipo de número trabaja el `Punto`. En este caso y respecto al "type erasure", ¿cuál es el tipo final tras la compilación?

Respuesta

En Java, es posible establecer restricciones en los parámetros de tipo utilizando la palabra clave `extends`. Esto permite limitar los tipos que se pueden usar como parámetros genéricos a aquellos que son subtipos de una clase o que implementan una interfaz específica. Por ejemplo, se puede restringir un parámetro genérico a tipos que sean subclases de `Number`.

Primera solución: Usar `Number` directamente como tipo de las coordenadas:

```java
public class Punto {
    private final Number x;
    private final Number y;

    public Punto(Number x, Number y) {
        this.x = x;
        this.y = y;
    }

    public Number getX() {
        return x;
    }

    public Number getY() {
        return y;
    }

    public double calcularDistanciaA(Punto otro) {
        double dx = x.doubleValue() - otro.getX().doubleValue();
        double dy = y.doubleValue() - otro.getY().doubleValue();
        return Math.sqrt(dx * dx + dy * dy);
    }
}
```

Segunda solución: Usar generics para reforzar el chequeo de tipos:

```java
public class Punto<T extends Number> {
    private final T x;
    private final T y;

    public Punto(T x, T y) {
        this.x = x;
        this.y = y;
    }

    public T getX() {
        return x;
    }

    public T getY() {
        return y;
    }

    public double calcularDistanciaA(Punto<T> otro) {
        double dx = x.doubleValue() - otro.getX().doubleValue();
        double dy = y.doubleValue() - otro.getY().doubleValue();
        return Math.sqrt(dx * dx + dy * dy);
    }
}
```

En la primera solución, se permite trabajar con cualquier subtipo de `Number`, pero no se refuerza el chequeo de tipos, lo que significa que las coordenadas podrían ser de tipos diferentes (por ejemplo, una `Integer` y otra `Double`). En la segunda solución, el uso de generics garantiza que ambas coordenadas sean del mismo tipo.

Respecto al "type erasure", en la solución con generics, el tipo genérico `T` se reemplaza por `Number` en tiempo de compilación, por lo que el tipo final tras la compilación es equivalente al de la primera solución.

10. Sobre las soluciones anteriores. Si bien ambas permiten trabajar con distintos tipos de número sin duplicar la clase `Punto`, reflexiona sobre el refuerzo del chequeo de tipos con generics. ¿Permiten ambas crear un punto con una coordenada de tipo entero y la otra coordenada de tipo real? ¿Qué tipo devuelve el

`getX` con la solucion sin generics y qué tipo devuelve el que tiene la solución con generics?

Respuesta

Ambas soluciones permiten trabajar con distintos tipos de números sin necesidad de duplicar la clase `Punto`. Sin embargo, hay diferencias importantes en cuanto al refuerzo del chequeo de tipos y la flexibilidad que ofrecen.

En la solución sin generics, las coordenadas se definen como `Number`, lo que permite crear un punto con una coordenada de tipo entero (`Integer`) y otra de tipo real (`Double`). Sin embargo, esto puede llevar a inconsistencias, ya que no hay garantía de que ambas coordenadas sean del mismo tipo. Además, el método `getX` devuelve un objeto de tipo `Number`, lo que puede requerir conversiones explícitas si se necesita un tipo más específico.

En la solución con generics, el uso de un parámetro de tipo `T` garantiza que ambas coordenadas sean del mismo tipo. Esto refuerza el chequeo de tipos en tiempo de compilación y evita errores relacionados con tipos inconsistentes. El método `getX` devuelve un objeto del tipo genérico `T`, lo que elimina la necesidad de realizar conversiones explícitas en la mayoría de los casos.

En resumen, la solución con generics proporciona un mayor nivel de seguridad de tipos y evita errores en tiempo de ejecución, pero no permite mezclar tipos diferentes para las coordenadas de un mismo punto.

11. Hagamos un ejemplo avanzado. El siguiente código, con interfaz `Punto`, que define un método `calcularDistanciaA(Punto p)`, junto con las implementaciones `Punto2D` y `Punto3D`. Añade generics para asegurarnos que la sobreescritura del método calcular distancia a otro `Punto` siempre es sobre un

`Punto` del mismo tipo, evitando `instanceof` y el downcasting.

Respuesta

El siguiente ejemplo utiliza generics para garantizar que el método `calcularDistanciaA` opere únicamente con puntos del mismo tipo, eliminando la necesidad de usar `instanceof` y downcasting:

```java
public interface Punto<T extends Punto<T>> {
    double calcularDistanciaA(T p);
}

public class Punto2D implements Punto<Punto2D> {
    private final double x, y;

    public Punto2D(double x, double y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public double calcularDistanciaA(Punto2D p) {
        double dx = x - p.x;
        double dy = y - p.y;
        return Math.sqrt(dx * dx + dy * dy);
    }
}

public class Punto3D implements Punto<Punto3D> {
    private final double x, y, z;

    public Punto3D(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }

    @Override
    public double calcularDistanciaA(Punto3D p) {
        double dx = x - p.x;
        double dy = y - p.y;
        double dz = z - p.z;
        return Math.sqrt(dx * dx + dy * dy + dz * dz);
    }
}
```

En este diseño, la interfaz `Punto` utiliza un parámetro genérico `T` que se extiende a sí mismo (`Punto<T>`). Esto asegura que cada implementación de `Punto` solo pueda calcular distancias con puntos del mismo tipo, eliminando la necesidad de comprobaciones en tiempo de ejecución.

12. Dado que `String` es subtipo de `Object`, ¿significa eso que `List<String>` es subtipo de `List<Object>`? ¿Y que `String[]` es subtipo de `Object[]`? Razona por qué la respuesta es diferente en cada caso y qué problema en tiempo de ejecución puede aparecer con los arrays. A partir de estos ejemplos, define qué significa que un tipo genérico sea **covariante**, **contravariante** o **invariante** respecto a su parámetro de tipo.

Respuesta

En Java, `String[]` es subtipo de `Object[]`, pero `List<String>` no es subtipo de `List<Object>`. Esto se debe a cómo se manejan los arrays y los genéricos en Java.

Los arrays en Java son **covariantes**, lo que significa que un array de un subtipo (`String[]`) puede ser asignado a una referencia de un supertipo (`Object[]`). Sin embargo, esto puede provocar errores en tiempo de ejecución. Por ejemplo:

```
Object[] objetos = new String[2];
objetos[0] = 42; // Error en tiempo de ejecución: ArrayStoreException
```

Por otro lado, los genéricos en Java son **invariantes**, lo que significa que `List<String>` no es subtipo de `List<Object>`. Esto se debe a que permitir tal asignación podría comprometer la seguridad de tipos en tiempo de compilación. Por ejemplo:

```
List<Object> objetos = new ArrayList<String>(); // No permitido
objetos.add(42); // Esto sería un problema si se permitiera
```

Definiciones:

- **Covariante**: Un tipo genérico `A<T>` es subtipo de `A<S>` si `T` es subtipo de `S`.
- **Contravariante**: Un tipo genérico `A<T>` es subtipo de `A<S>` si `S` es subtipo de `T`.
- **Invariante**: Un tipo genérico `A<T>` no es subtipo ni supertipo de `A<S>`, incluso si `T` es subtipo de `S`.

13. Java permite recuperar covarianza y contravarianza en tipos genéricos de forma controlada mediante **wildcards**. ¿Qué es un wildcard (`?`)? Muestra la diferencia entre `List<? extends T>` y `List<? super T>`, indicando en qué casos se usa cada uno. Pon dos ejemplos: (i) un método que reciba una lista de números y calcule su suma, usando `? extends`; (ii) un método que reciba una lista y le añada varios números enteros, usando `? super`.

Respuesta

Un wildcard (`?`) es un comodín que representa un tipo desconocido en una declaración genérica. Se utiliza para flexibilizar el uso de tipos genéricos en métodos y clases.

- `List<? extends T>` : Permite leer elementos de la lista, pero no añadir elementos (excepto `null`). Se utiliza cuando el método necesita procesar elementos de la lista sin modificarla.
- `List<? super T>` : Permite añadir elementos a la lista, pero las operaciones de lectura solo garantizan que los elementos sean de tipo `Object` . Se utiliza cuando el método necesita modificar la lista.

Ejemplo con `? extends` :

```java
public static double sumarLista(List<? extends Number> lista) {
    double suma = 0;
    for (Number num : lista) {
        suma += num.doubleValue();
    }
    return suma;
}
```

Ejemplo con `? super` :

```java
public static void agregarEnteros(List<? super Integer> lista) {
    lista.add(1);
    lista.add(2);
    lista.add(3);
}
```

En el primer caso, se puede pasar una lista de cualquier subtipo de `Number` (como `List<Integer>` o `List<Double>`), pero no se pueden añadir elementos. En el segundo caso, se puede pasar una lista de cualquier supertipo de `Integer` (como `List<Number>` o `List<Object>`), y se pueden añadir elementos de tipo `Integer`.